CS2102 MINI PROJECT

IITPSx Processor Assembler & Emulator

Detailed Technical Report

A two-pass assembler, a machine-code emulator, and assembly-language test programs implemented in C.

Project	IITPSx Processor Mini Project — 2026
Main source files	asm.c, emu.c
Build system	GCC and Makefile
Test examples	Assembler error tests, SET, mul/div, division-by-zero, bubble sort
Student name / ID	Add your name and user ID before submission

Prepared from the supplied mini-project specification, C source files, assembly programs, and saved test evidence.

1. Executive Summary

This project builds a small software toolchain for the IITPSx processor. The assembler reads human-readable assembly source and translates it into 32-bit instruction words stored in a binary object file. The emulator then loads that object file and simulates the processor's execution. A listing file and log file make the translation process easier to inspect and debug.

The supplied implementation is organized around two C programs: **asm.c** and **emu.c**. The assembler performs two passes so that labels can be defined and referenced before their location is known. The emulator models the IITPSx accumulator-style stack registers, program counter, stack pointer, memory, instruction decoding, branches, and runtime errors. The solution also adds the required **mul** and **div** instructions and includes a bubble-sort assembly program.

Important evidence in the solution archive includes successful assembly listings, logs for malformed assembly inputs, a multiplication/division run ending with A = 21, a division-by-zero runtime error, and a memory dump showing the bubble-sort array in ascending order.

Project flow

1. Source	2. Assemble	3. Object file	4. Execute	5. Evidence
program.asm	asm.c / asm	program.o	emu.c / emu	.lst, .log, .dump

The assembler and emulator are separate programs. The object file is the interface between them.

2. Project Requirements and Goals

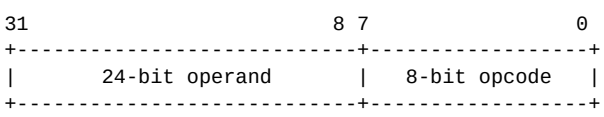
The original brief asks for a two-pass assembler, test programs, a bubble-sort program, and an emulator with sufficient functionality to execute the machine code. It also specifically requires the two additional arithmetic instructions, mul and div, including division-by-zero detection.

- **Assembler:** parse line-based assembly; support labels and numeric operands; encode opcodes and operands; diagnose common source errors; create a binary object file, listing, and log.
- **Pass handling:** discover label addresses in Pass 1, then resolve operands and emit code in Pass 2.
- **Assembly programs:** assemble provided examples and additional valid/invalid test programs; implement and test bubble sort.
- **Emulator:** load the object file, decode and execute instructions, maintain processor state and memory, and stop on HALT or report runtime errors.
- **Build quality:** compile with GCC using strict C89/pedantic warning flags and provide reproducible build commands.

3. IITPSx Architecture and Instruction Encoding

The specification describes four 32-bit processor registers: A and B, which form an internal stack-like accumulator arrangement, plus the program counter (PC) and stack pointer (SP). The emulator also models memory as an array of 65,536 integer words.

Instruction word layout



The lower 8 bits hold the opcode. The upper 24 bits hold the operand, interpreted as a signed two's-complement value where applicable. Instructions such as add and mul have no explicit operand; instructions such as ldc, adc, ldl, branches, and call carry an operand. The assembler's object writer stores each 32-bit word as four bytes in little-endian order, and the emulator reconstructs the same word when loading the object file.

Register / item	Purpose in this implementation
A	Primary accumulator; many instructions read or update A.
B	Secondary accumulator / stack partner; arithmetic instructions commonly combine B and A.
PC	Address of the next instruction; incremented before the instruction action.
SP	Stack pointer used by local load/store and stack-adjust instructions.
Memory	Word-addressed array used for program instructions and data.

Selected instruction behavior

Instruction	Opcode	Effect / purpose
ldc value	0	B := A; A := value
adc value	1	A := A + value
ldl / stl	2 / 3	Load/store at memory[SP + offset]
ldnl / stnl	4 / 5	Load/store using A as the base address
add / sub	6 / 7	A := B + A / A := B – A
call / return	13 / 14	Procedure call and return using PC and accumulator state
brz / brlz / br	15 / 16 / 17	Conditional or unconditional relative branch
HALT	18	Stops execution
mul / div	19 / 20	A := B × A; or A := B ÷ A (error if A is zero)

4. Assembler Implementation — asm.c

The assembler translates assembly text into object code and creates human-readable artifacts for inspection. Its instruction table stores each mnemonic, its opcode, and whether an operand is required. This central table makes the parser's validation and encoding rules easier to maintain.

4.1 Pass 1 — symbol discovery and address counting

- The source file is read using fgets and each line is retained so it can be processed twice.
- Comments beginning with a semicolon are removed from the active text; blank lines and comment-only lines are ignored.
- A label definition such as `loop:` is recorded with the current program counter (PC) value.
- Each valid instruction or data word advances the assembler's address counter, allowing later labels to receive correct addresses.
- The SET pseudo-instruction can assign a label a specified numeric value rather than the current PC.

4.2 Pass 2 — resolution and code generation

- The source is scanned again using the symbol table built during Pass 1.
- Numeric operands are parsed with strtol, allowing decimal, hexadecimal, and octal forms where valid.
- Label operands are replaced with their recorded values. For branch and call instructions, the assembler computes a relative displacement as target address − (current instruction address + 1).
- The instruction word is formed by placing the opcode in the lowest eight bits and the operand in the upper bits. Data directives emit the data value itself.
- The assembler writes the object file in binary mode and produces a listing (.lst) and log (.log). If errors are detected, it reports them and does not emit a successful program listing/object payload.

4.3 Source diagnostics

The code checks for common errors including duplicate labels, invalid label names, unknown mnemonics, undefined labels, missing operands, unexpected operands, invalid extra text, invalid SET usage, and invalid SET values. Labels that are defined but never referenced are reported as warnings rather than fatal errors.

4.4 Output files

Artifact	Purpose
.o	Binary object code: four bytes per 32-bit machine word.
.lst	Address, encoded word, and original source statement for each generated word.
.log	Assembly status, errors/warnings, and symbol-table label values.

The listing is especially useful for debugging because it lets the reader compare an assembly mnemonic and operand with the hexadecimal machine word produced by the assembler.

5. Emulator Implementation — emu.c

The emulator loads the binary object file, initializes A, B, PC, and SP to zero, and starts execution at address zero. Each instruction is fetched from memory, the PC is incremented, the low byte is decoded as the opcode, and the upper bits are treated as the operand. A switch statement implements the instruction behavior.

5.1 Execution cycle

```
Load object file
      |
Initialize A, B, PC, SP and step counter
      |
Fetch word at memory[PC]
      |
Increment PC; decode opcode and operand
      |
Execute instruction / update registers and memory
      |
HALT? ---- yes ---> print state and write memory dump
  |
  no
  +----> validate PC and continue
```

5.2 Runtime checks

- **Object loading:** rejects an object file that ends with an incomplete four-byte word or exceeds the configured memory capacity.
- **Address validation:** checks PC and the addresses used by local and non-local load/store instructions.
- **Opcode validation:** unknown opcodes produce a runtime error.
- **Division safety:** div checks whether A is zero before performing signed integer division.
- **Runaway execution:** a maximum instruction-step count prevents an infinite loop from running forever.

After execution, the emulator prints the final values of A, B, PC, SP, and the number of executed instructions. It also writes a memory dump for the loaded program words. These outputs make it possible to verify expected results and investigate failures.

6. Test Strategy and Results

The solution archive contains evidence files in an evidence/ directory. The table below summarizes the intent of the tests and the observed result recorded in the supplied logs/dumps. Tests that intentionally contain invalid assembly should fail to assemble; that failure is the expected behavior, not a defect.

Test / artifact	Purpose	Observed evidence
test1	Valid syntax, signed constants, labels, branches, forward reference and data.	Assembly succeeded; warning for unused label; listing contains encoded words and resolved labels.
test2	Negative test for duplicate labels, undefined labels, invalid numeric operand, missing/unexpected operands and unknown mnemonics.	Assembler log records multiple diagnostics; no successful listing is produced.
test3	SET pseudo-instruction.	Assembly succeeded; val = 75 and val2 = 66 appear in the label log; listing encodes ldc/adc operands.
test4	Provided larger sample program.	Assembly succeeded and listing/log were generated.
test5	Loop and arithmetic sample.	Assembly succeeded; listing shows branch and arithmetic instructions.
test6	Unused-label warnings and control-flow sample.	Assembly succeeded; unused labels are warned about.
test7	Additional malformed input.	Assembler log reports invalid label, duplicate label, missing operand, unknown mnemonic, undefined label and SET errors.

Test / artifact	Purpose	Observed evidence
test08_muldiv	Required multiplication and division extension.	Final A = 21 (0x15), B = 42; 5 instructions executed before HALT.
test09_divzero	Runtime division-by-zero handling.	Emulator reports 'division by zero at PC 2' and returns an error.
bubble_sort.asm	Sort five values: 5, 1, 4, 2, 8.	Final array words in the memory dump are 1, 2, 4, 5, 8.

6.1 Detailed arithmetic test

```
ldc 6
ldc 7
mul        ; A = 42
ldc 2
div        ; A = 21
HALT
```

The emulator evidence records A = 21 (decimal), hexadecimal 0x00000015, and B = 42. This matches the expected result: 6 × 7 = 42, then 42 ÷ 2 = 21.

6.2 Division-by-zero test

```
ldc 42
ldc 0
div
HALT
```

The emulator correctly stops at the div instruction and reports a division-by-zero runtime error. This confirms the required guard is present in the emulator’s execution path.

6.3 Bubble-sort verification

The assembly program compares adjacent values and swaps them when they are out of order. The provided version unrolls the compare-and-swap operations to keep the control flow explicit. The initial values are 5, 1, 4, 2, 8. In the saved memory dump, the array begins at word address 0x00C0 and contains 1, 2, 4, 5, 8 in ascending order; the preceding word at 0x00BF is the temporary storage location.

7. Build and Run Instructions

Run these commands from the project directory in a terminal where GCC is available. In Windows, the same commands can be used from a suitable MinGW/GCC terminal; on Linux/macOS, use the system terminal.

7.1 Compile

```
gcc -std=c89 -pedantic -W -Wall -Wpointer-arith -Wwrite-strings -Wstrict-prototypes asm.c -o asm
gcc -std=c89 -pedantic -W -Wall -Wpointer-arith -Wwrite-strings -Wstrict-prototypes emu.c -o emu
```

Alternatively, use the included Makefile:

```
make
```

7.2 Assemble a program

```
./asm test08_muldiv.asm test08_muldiv.o test08_muldiv.lst test08_muldiv.log
```

The four arguments after the executable are the source file, output object file, listing file, and log file. On Windows terminals, the executable names may be asm.exe and emu.exe.

7.3 Execute the object file

```
./emu test08_muldiv.o test08_muldiv.dump
```

The emulator prints register state and instruction count to the terminal and writes the memory dump to the specified file. Check that the arithmetic test finishes with A = 21. For test09_divzero, a runtime error is expected.

7.4 Example project folder structure

```
MiniProject/
  asm.c
  emu.c
  Makefile
  README.txt
  claims
  bubble_sort.asm
  test08_muldiv.asm
  test09_divzero.asm
  evidence/
    test08_muldiv.o
    test08_muldiv.lst
    test08_muldiv.log
    test08_muldiv.dump
    test08_muldiv.emu.log
    test09_divzero.*
    bubble_sort.*
```

The evidence folder should contain the test source files and outputs needed to reproduce and inspect the reported results. Keep source files separate from generated artifacts so it is clear which files are edited and which are produced by the build.

8. Key Design Decisions

Decision	Reason / benefit
Instruction table	Keeps mnemonic, opcode, and operand requirements together and reduces repeated special cases.
Two-pass assembly	Allows forward references and label-based branch offsets to be resolved.
Binary object format	Provides a compact machine-code file that the emulator can load directly.
Listing and log outputs	Make encoding, labels, warnings, and errors easier to inspect during debugging.
Explicit runtime checks	Prevent invalid addresses, unknown instructions, division by zero, and runaway loops from failing silently.
Saved test evidence	Demonstrates both successful assembly/execution and expected error handling.

9. Limitations and Submission Checks

The report describes the behavior evidenced by the supplied source and output files. Before final submission, review the following items because they affect grading and reproducibility:

- **Replace placeholders:** asm.c, emu.c, and claims currently contain “YOUR NAME” and “YOUR USER ID”. Replace these with your actual details.
- **Confirm compiler output:** compile both programs with the specified strict flags and resolve any warnings or errors shown by your local compiler.
- **Retain test sources:** the submission brief requests test assembly files alongside corresponding logs and listings. Ensure all required test inputs are included, not only their output artifacts.
- **Check the claims file:** only claim features you can explain and demonstrate. The marker may ask for a detailed explanation of the code structure and instruction semantics.
- **Be precise about the emulator scope:** this emulator implements the instruction behaviors coded in emu.c; it is a teaching simulator, not a hardware processor or a complete industrial-grade execution environment.
- **Review portability:** the project is intended to be checked with C89 flags. Validate compiler behavior on the exact environment used for submission.

10. Conclusion

This mini-project connects the main stages of a simple processor toolchain. The assembler parses assembly text, records symbols in Pass 1, resolves labels and emits machine words in Pass 2, and produces a binary object file with supporting listings and logs. The emulator loads that file, decodes and executes the instructions, maintains processor state, and detects important runtime errors.

The saved evidence demonstrates successful assembly of valid programs, diagnosis of invalid source, SET handling, the mul/div extension, division-by-zero detection, and a bubble-sort result of 1, 2, 4, 5, 8. Together, the C source, assembly programs, Makefile, and evidence files form a reproducible educational implementation of the IITPSx assembler/emulator workflow.

Appendix A — Quick command reference

Action	Command
Build both tools	make
Build assembler only	gcc -std=c89 -pedantic -W -Wall asm.c -o asm
Build emulator only	gcc -std=c89 -pedantic -W -Wall emu.c -o emu
Assemble test	./asm test08_muldiv.asm test08_muldiv.o test08_muldiv.lst test08_muldiv.log
Run test	./emu test08_muldiv.o test08_muldiv.dump
Clean generated files	make clean

Appendix B — File responsibility map

File	Responsibility
asm.c	Two-pass source parser, label table, instruction validation, machine-word encoding, object/listing/log generation.
emu.c	Object loading, instruction decoding, processor-state simulation, runtime checks, state output and memory dump.
Makefile	Build rules for asm and emu using strict compiler flags; clean rule for generated files.
bubble_sort.asm	Assembly-language sorting program and data values.
test08_muldiv.asm	Successful test for multiplication and division instructions.
test09_divzero.asm	Negative runtime test for division by zero.
evidence/	Saved object files, listings, logs, emulator logs, and memory dumps used to inspect test outcomes.
README.txt / claims	Build guidance, instruction-format notes, submission checklist, and declared implementation claims.

End of report